MADHAV INSTITUTE OF TECHNOLOGY & SCIENCE, GWALIOR (M.P.), INDIA

Deemed University

(Declared under Distinct Category by Ministry of Education, Government of India)

NAAC ACCREDITED WITH A++ GRADE

PRACTICAL FILE

ON

SOFTWARE ENGINEERING PRINCIPLES LAB

(68251209)

Submitted By:

Surya Pratap Singh

[MCCA25O1050]

Faculty Mentors:

Dr. Parul Saxena, *Assistant Professor*

Ms. Kajal Chaurasiya, Research Assistant

Department of Computer Science and Engineering

Madhav Institute of Technology & Science

Gwalior-474005(M.P) Estb.1957

Jan-June 2026

Table of Contents

PROJECT 1: Design of Canteen Management System

1. INTRODUCTION

1.1 Purpose

Design of Canteen Management System is developed to improve the overall functioning of a college canteen by introducing a structured and digital approach. In most colleges, canteen operations are still handled manually, which often leads to long queues, incorrect billing, and poor inventory management. These issues not only affect efficiency but also reduce user satisfaction.

The main purpose of this system is to simplify and organize the complete process of ordering, billing, and management using a single platform. It allows students and staff to place orders easily, reduces waiting time, and ensures that records are maintained properly.

This system mainly focuses on:

i. Making the ordering process faster and more convenient
ii. Reducing manual errors in billing and transactions
iii. Keeping track of stock and avoiding shortages
iv. Providing useful reports to the admin for better decisions
v. Collecting feedback to improve food quality and service

1.2 Scope

The scope of this project covers all major activities involved in running a canteen, starting from user login to order completion and report generation. It is designed in such a way that it can be easily used in any college or institutional environment.

The system includes the following features:

i. **User Management:** Different users like students, staff, and admin can log in securely and access features based on their role.
ii. **Menu and Order Management:** Users can view available items, select them, and place orders without standing in queues. The system also shows item availability.
iii. **Billing and Payment:** Bills are generated automatically, and users can choose different payment methods like cash or online payment.
iv. **Inventory Management:** The system keeps track of stock levels and helps in avoiding situations where items go out of stock suddenly.
v. **Admin Control:** The admin can manage menu items, monitor orders, and view daily reports.
vi. **Feedback System:** Users can give ratings and comments, which help in improving the service.

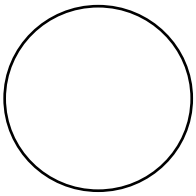
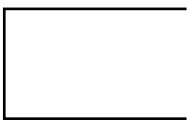
2. DATA FLOW DIAGRAM (DFD)

2.1 Definition

A Data Flow Diagram (DFD) is used to show how data moves inside a system. It explains where the data comes from, how it is processed, and where it is stored. Compared to other diagrams, DFD is easy to understand because it focuses only on the flow of data and not on technical details.

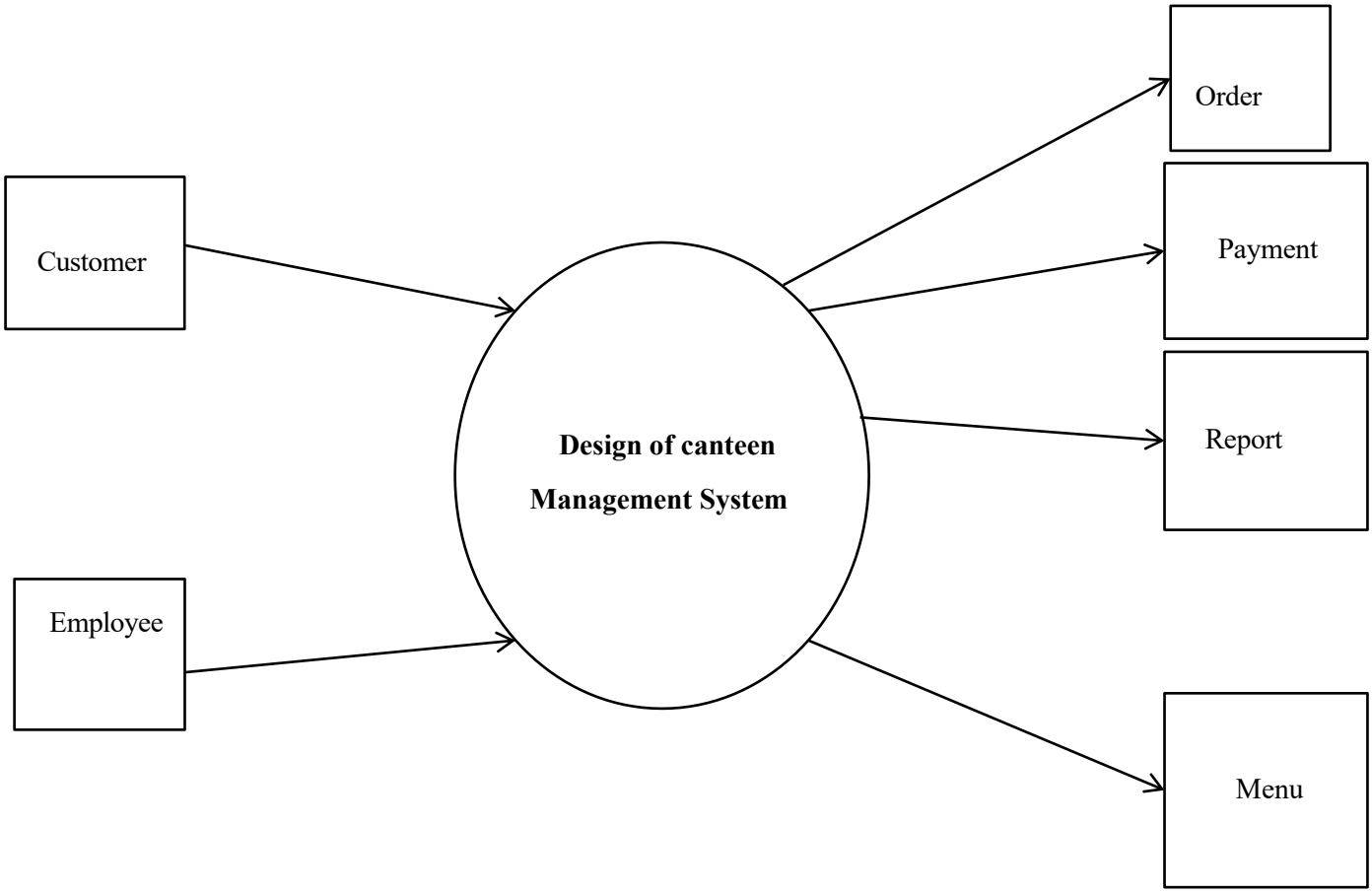
In the Canteen Management System, DFD helps in understanding how an order moves from the user to the staff, how payment is processed, and how data is stored for future use.

2.2 Symbols Used

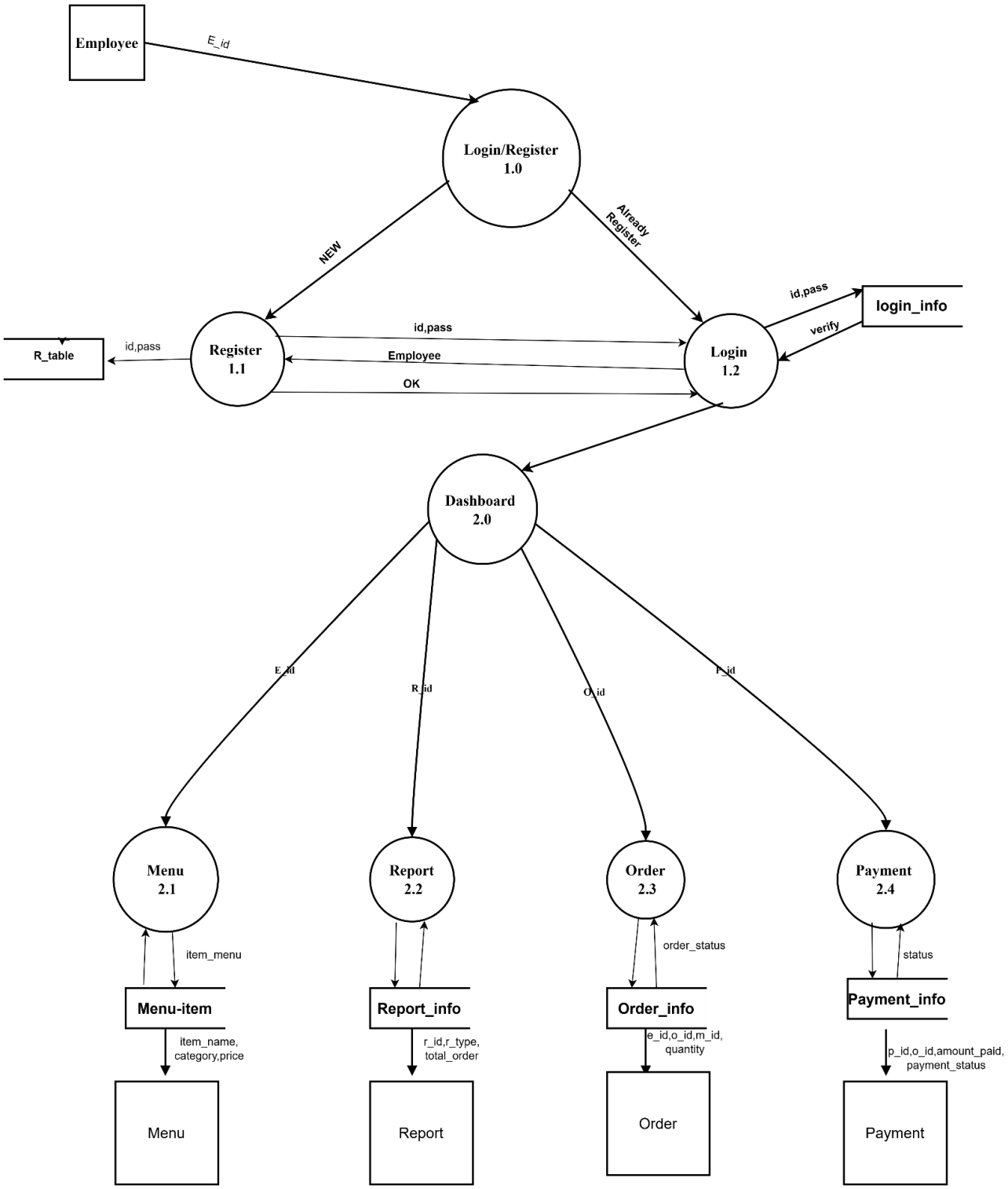
S. No	Symbol	Name	Description
1		Source/Destination	Represents a person or system that gives data to the system or receives data from it.
2		Process	Represents a process or activity that takes input data, processes it, and produces meaningful output.
3		Table	Represents a place where data is saved and stored.
4		Direction/Flow	Represents the flow of data between different parts of the system.

2.3 Example

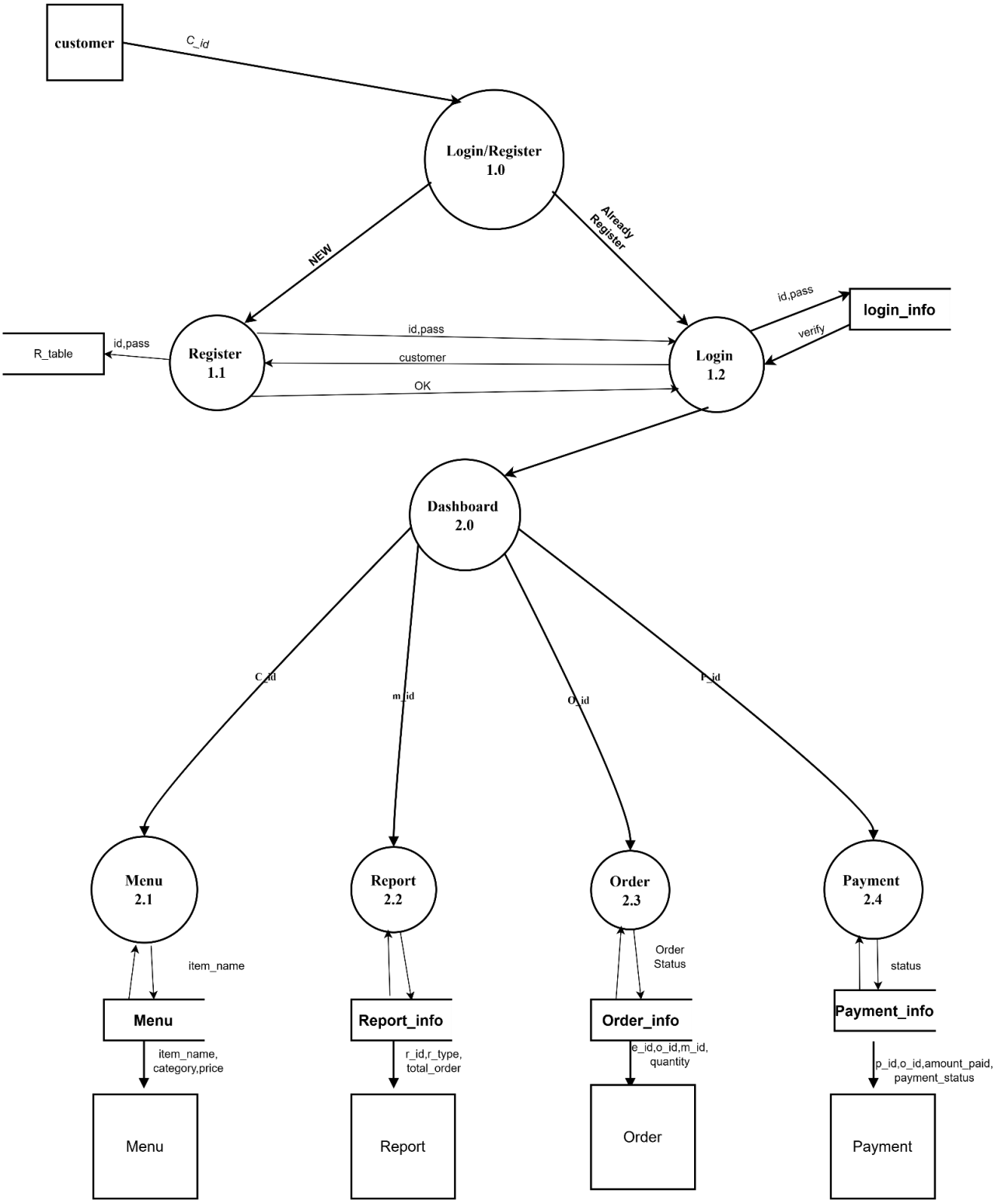
2.3.1 DFD Level 0



2.3.2 DFD Level 1 for Employee



2.3.3 DFD Level 1 for Customer

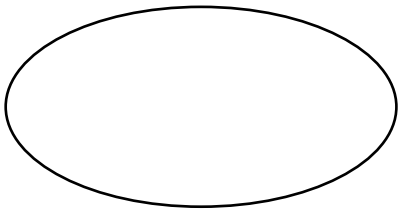
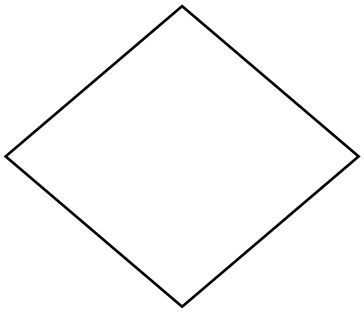
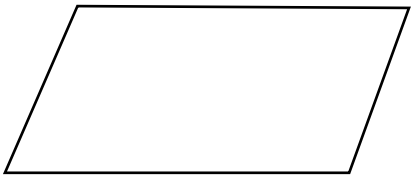


3. Flow Chart

3.1 Definition

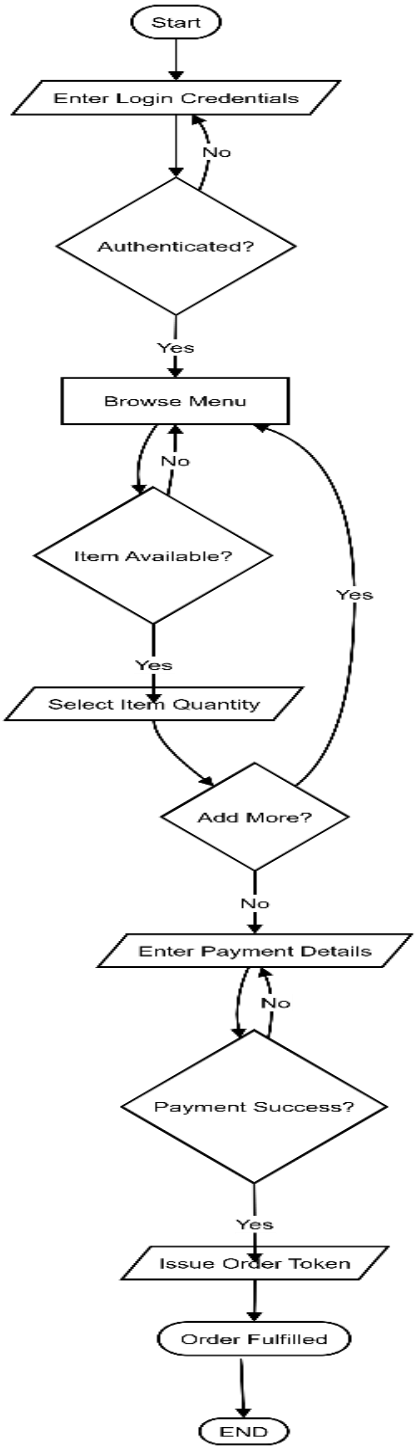
A flowchart is a graphical representation of a process or algorithm that helps in understanding the flow of steps visually.

3.2 Symbols Used

S. No	Symbol	Name	Description
1		Terminal/Terminator	Represents the beginning or end of a process, indicating where the flow starts or stops.
2		Process	Represents a step where an action, operation, or calculation is performed.
3		Decision	Represents a point where a condition is checked, resulting in different possible paths such as Yes or No.
4		Data or Input/Output	Represents the input of data into the system or the output of information from the system.

5		Flow Arrow	Shows the direction of flow in a process, connecting different steps in the flowchart.

3.3 Example



4. Unified Modeling Language (UML)

4.1 Definition

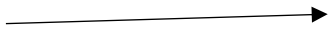
UML (Unified Modeling Language) is a standard visual language used to design and represent software systems using diagrams. It helps in understanding both the structure and behavior of a system.

4.2 Types of UML Relationships

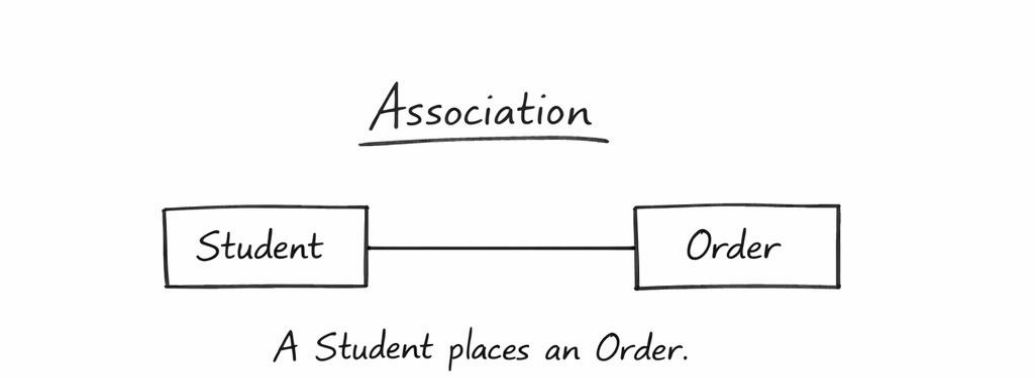
4.2.1 Association:

Association is a simple relationship between two classes that shows how they are connected or interact with each other. It represents a "uses" relationship and does not involve ownership.

Symbol:



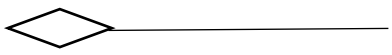
Example:



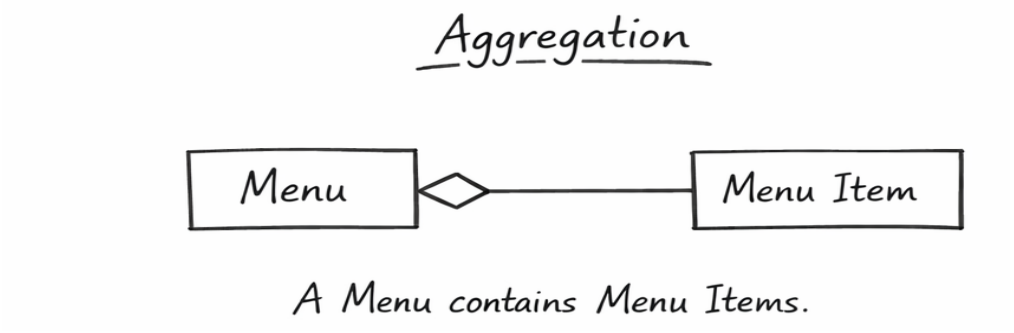
4.2.2 Aggregation:

Aggregation is a weak "has-a" relationship where one class contains another, but both can exist independently.

Symbol:



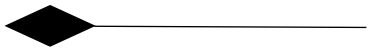
Example:



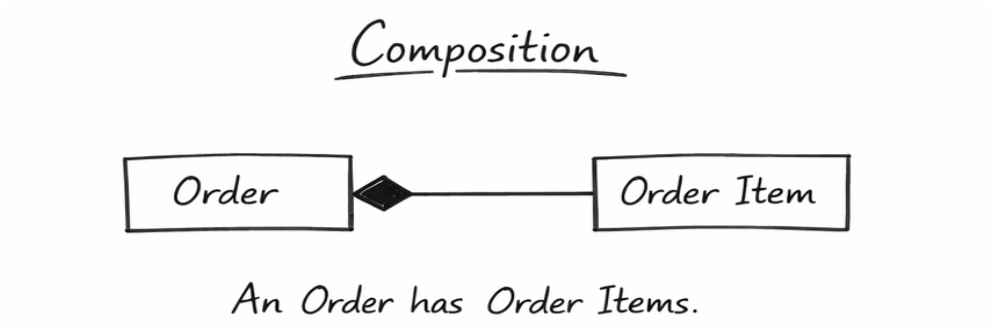
4.2.3 Composition

Composition is a strong “has-a” relationship where one class fully owns another, and the part cannot exist without the whole.

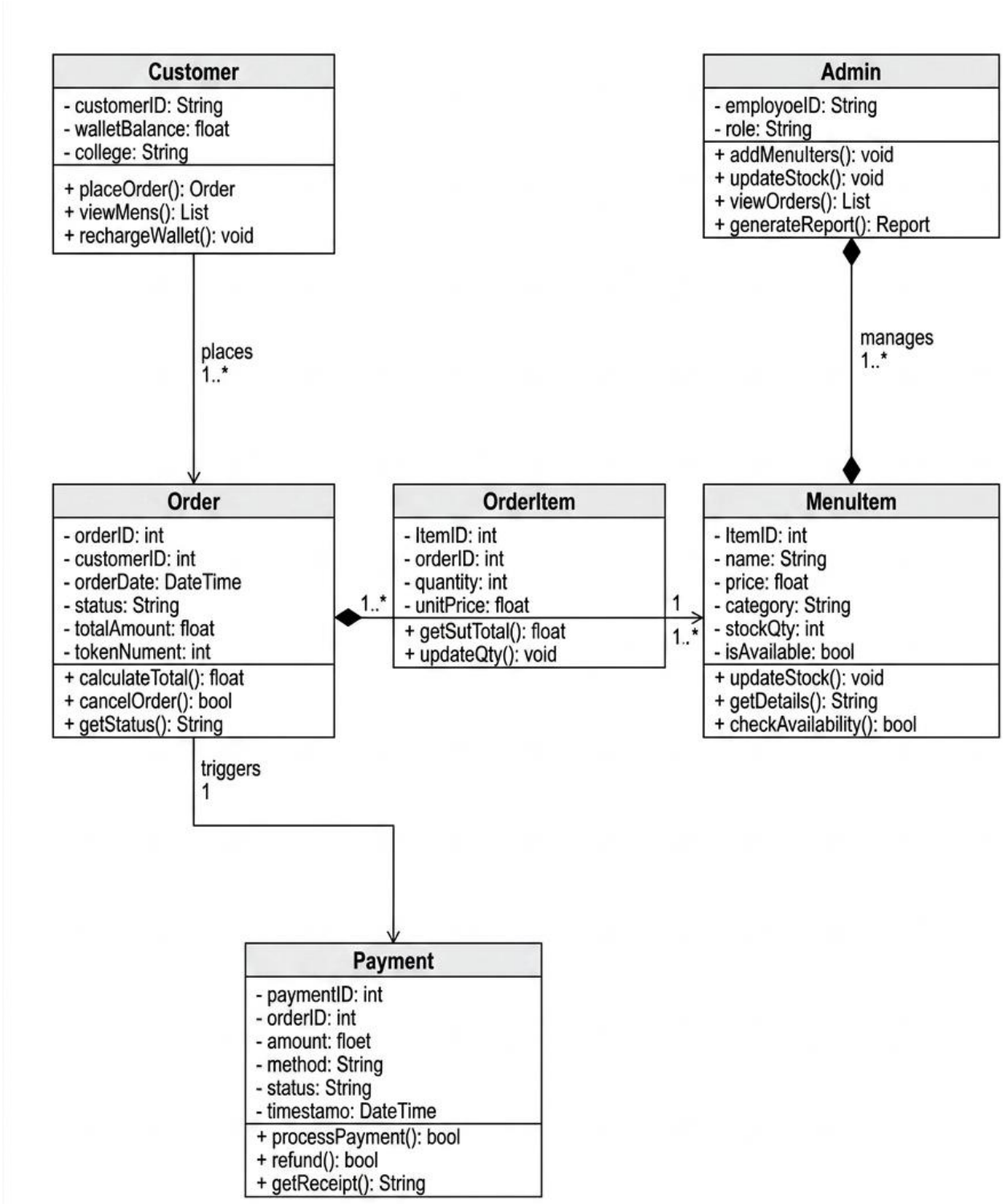
Symbol:



Example:



4.3 Class Diagram



5. ENTITY - RELATIONSHIP (ER) DIAGRAM\

5.1 Definition

An Entity Relationship Diagram (ERD) is a graphical tool used in database design to represent the structure of data and the relationships between different entities. It helps in visually understanding how data is organized, stored, and connected within a system.\

5.2 Components of ER Diagram

5.2.1 Entity

An entity represents any real-world object about which data is stored, such as a Student or Order.

Strong Entity: Has its own unique identity (e.g., Student)

Weak Entity: Depends on another entity for identification (e.g., OrderItem)

5.2.2 Attributes

Attributes describe the details or characteristics of an entity.

Simple: Cannot be further divided (Age)

Composite: Can be split into parts (Name → First Name, Last Name)

Multivalued: Can have more than one value (Phone Numbers)

Derived: Calculated from other attributes (Total Bill)

Key Attribute: Uniquely identifies each record (ID)

5.2.3 Relationship

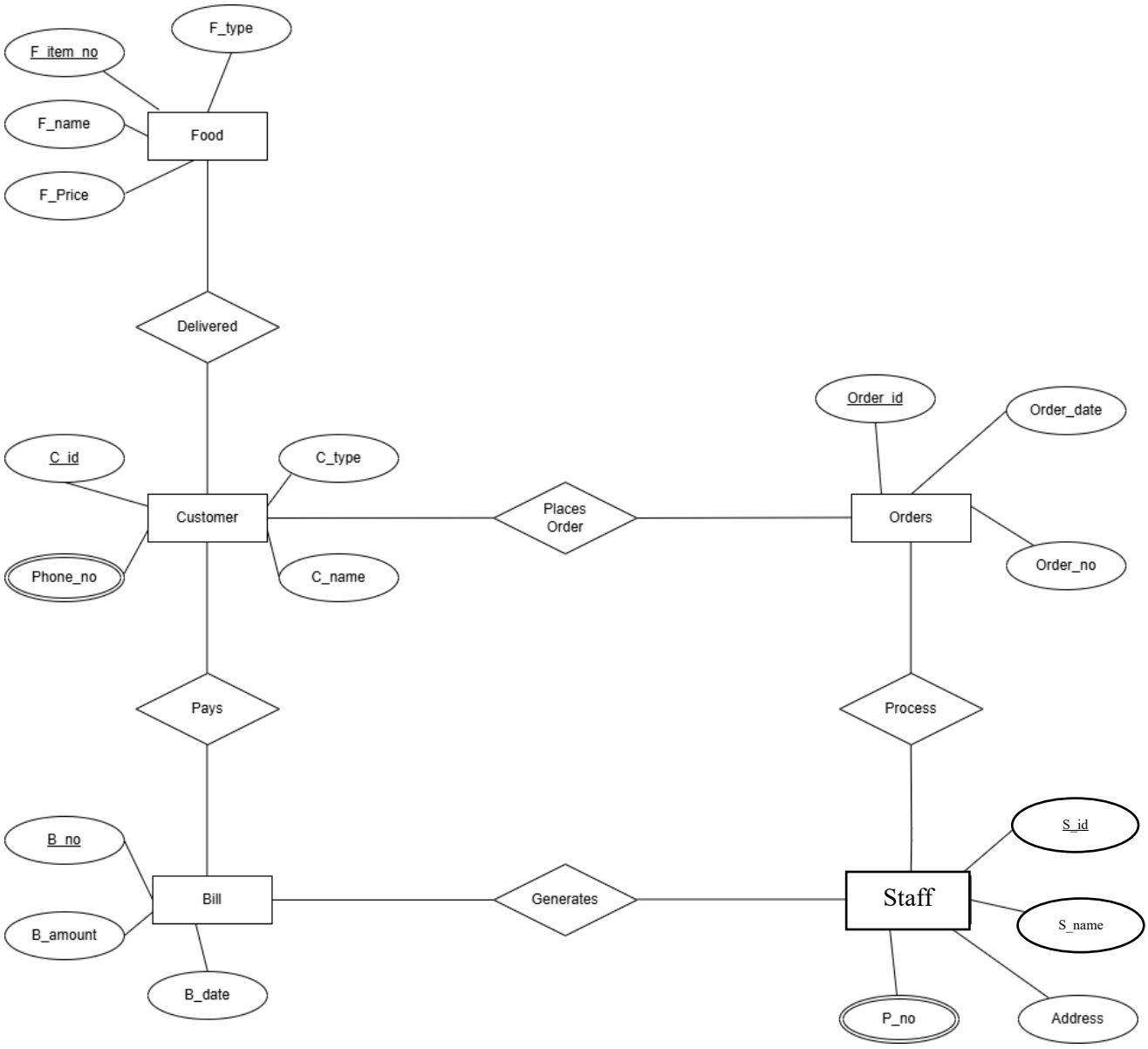
A relationship shows how different entities are connected within the system.

One-to-One (1:1): One entity is linked to one entity

One-to-Many (1:M): One entity is linked to multiple entities

Many-to-Many (M:N): Multiple entities are linked to multiple entities

5.3 Example


F_type
F_item_no
F_name
Food
F_Price
Delivered
Order_id
Order_date
C_id
C_type
Customer
Places Order
Orders
Phone_no
C_name
Order_no
Pays
Process
B_no
S_id
Bill
Generates
Staff
B_amount
S_name
B_date
P_no
Address

6. TEST CASE

6.1 Definition

A test case is a defined set of conditions, inputs, and steps used to check whether a software system is working correctly. It includes input data, expected results, and actual results, which help in verifying if the system meets the required specifications. Test cases are useful for identifying errors and ensuring proper system behavior.

6.2 Components

a. Test ID

It is a unique identification number or code given to each test case. It helps in easily tracking and managing test cases. Example: TC01, TC02.

b. Input Data

These are the values or conditions given to the software during testing. Input data is used to perform the test. Example: Username = “Admin”, Password = “1234”.

c. Expected Output

This is the result that should be produced if the system is working correctly. It is defined before testing starts.

d. Actual Output

This is the result that is actually produced when the test case is executed in the system.

6.3 Example

Test Case ID	Section	Element	Test Data	Expected Result	Actual Result
1	Admin Login	Username, Password	No Data	Show Fill all Field	Passed

			admin@123 , ****	Invalid Credentials	Passed
			admin@canteen ,****	Login Successful	Passed
2	User Login	Email, Password	Empty Feilds	Show Error	Passed
			naman@gmail , ****	Invalid Email	Passed
			naman@gmail.com , ****	Login Successful	Passed
3	Registration	Name, Email, Phone	No Data	Please fill out this field.	Passed
			Surya Pratap, Surya,****	Invalid Detail	Passed
			Surya Pratap ,suryapr2017@gmail.com , ****	Successfully Registered	Passed
4	Food Ordering	Item Quantity	No Item Selected	Show Warning	Passed
			(-1)	Invalid Input	Passed
			1	Order Placed	Passed
5	Payment	Payment Method	No Selection	Show Error	Passed

			abc@upi	Failed Transaction	Passed
			Cash	Payment Successfully	Passed

7. CONCLUSION

The development of the **Canteen Management System** has successfully addressed many problems associated with traditional canteen operations. By using a modern technology stack such as the MERN stack (MongoDB, Express, React, and Node.js), the system provides a smooth and responsive experience that can be accessed easily on both desktops and mobile devices.

With features like digital ordering, real-time order status updates, and proper inventory tracking, the system removes common issues such as long queues, billing errors, and lack of coordination between staff and users. The inclusion of a simple feedback system also helps in improving food quality and service based on user responses.

In addition, the system makes the entire process more transparent and organized, allowing the admin to monitor daily activities and make better decisions using available data.

In conclusion, this project contributes to a more efficient and user-friendly canteen environment, and it can be further improved in the future by adding advanced features like demand prediction and mobile notifications.

PROJECT 2: Design of Smart Parking Management System

1. INTRODUCTION

1.1 Purpose

The Design of Smart Parking Management System is developed to modernize and streamline the management of parking facilities in colleges, malls, hospitals, and other public spaces. Conventional parking systems still rely heavily on manual ticketing, cash collection, and human-directed slot allocation, leading to traffic congestion, revenue leakage, and poor user experience.

The primary purpose of this system is to automate and digitize every aspect of parking — from slot detection and vehicle entry to billing and report generation — using a centralized platform. It enables both drivers and parking operators to interact with the system efficiently, minimizing manual intervention.

This system mainly focuses on:

a. Automating vehicle entry and exit using QR codes or number plate detection
b. Providing real-time visibility of available parking slots
c. Reducing manual billing errors through automatic fare calculation
d. Enabling digital payment options such as UPI, card, and wallet
e. Generating daily, weekly, and monthly reports for the admin
f. Collecting user feedback to continuously improve parking services

1.2 Scope

The scope of this project covers all major activities involved in running a smart parking facility, from vehicle registration at the entry gate to slot allocation, billing, exit processing, and report generation. It is designed to be easily deployable in any institutional or commercial parking environment.

The system includes the following features:

a. **User Management:** Drivers, operators, and administrators can log in securely and access features appropriate to their role.
b. **Slot Management:** The system displays real-time slot availability categorized by vehicle type (two-wheeler, four-wheeler, differently-abled) and allows users to reserve slots in advance.

c. **Vehicle Entry and Exit:** The system records vehicle entry with timestamp and allocated slot number; on exit it automatically calculates parking duration and fare.
d. **Billing and Payment:** Bills are generated automatically based on the vehicle type and duration. Multiple payment modes including cash, card, and online wallet are supported.
e. **Admin Control:** Administrators can manage slot configurations, monitor live occupancy, oversee transactions, and generate financial reports.
f. **Feedback System:** Users can submit ratings and comments after each parking session, helping management to improve service quality.

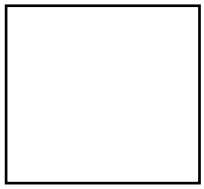
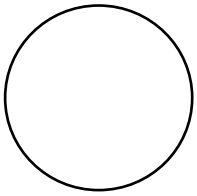
2. DATA FLOW DIAGRAM (DFD)

2.1 Definition

A Data Flow Diagram (DFD) is used to show how data moves inside a system. It explains where the data comes from, how it is processed, and where it is stored. Compared to other diagrams, DFD is easy to understand because it focuses only on the flow of data and does not include implementation details.

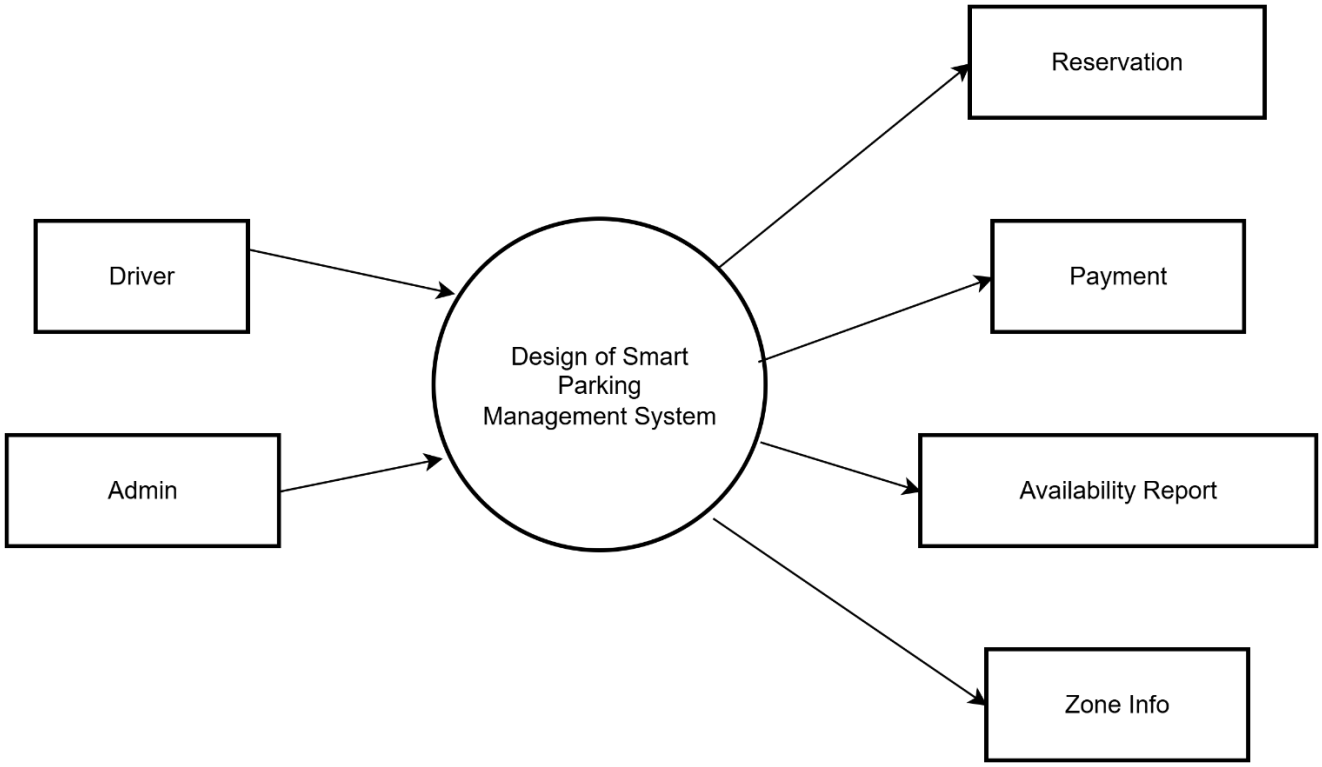
In the Smart Parking Management System, DFD helps in understanding how a vehicle entry request moves from the driver, gets processed by the system, triggers slot allocation, and finally produces a billing receipt on exit.

2.2 Symbols Used

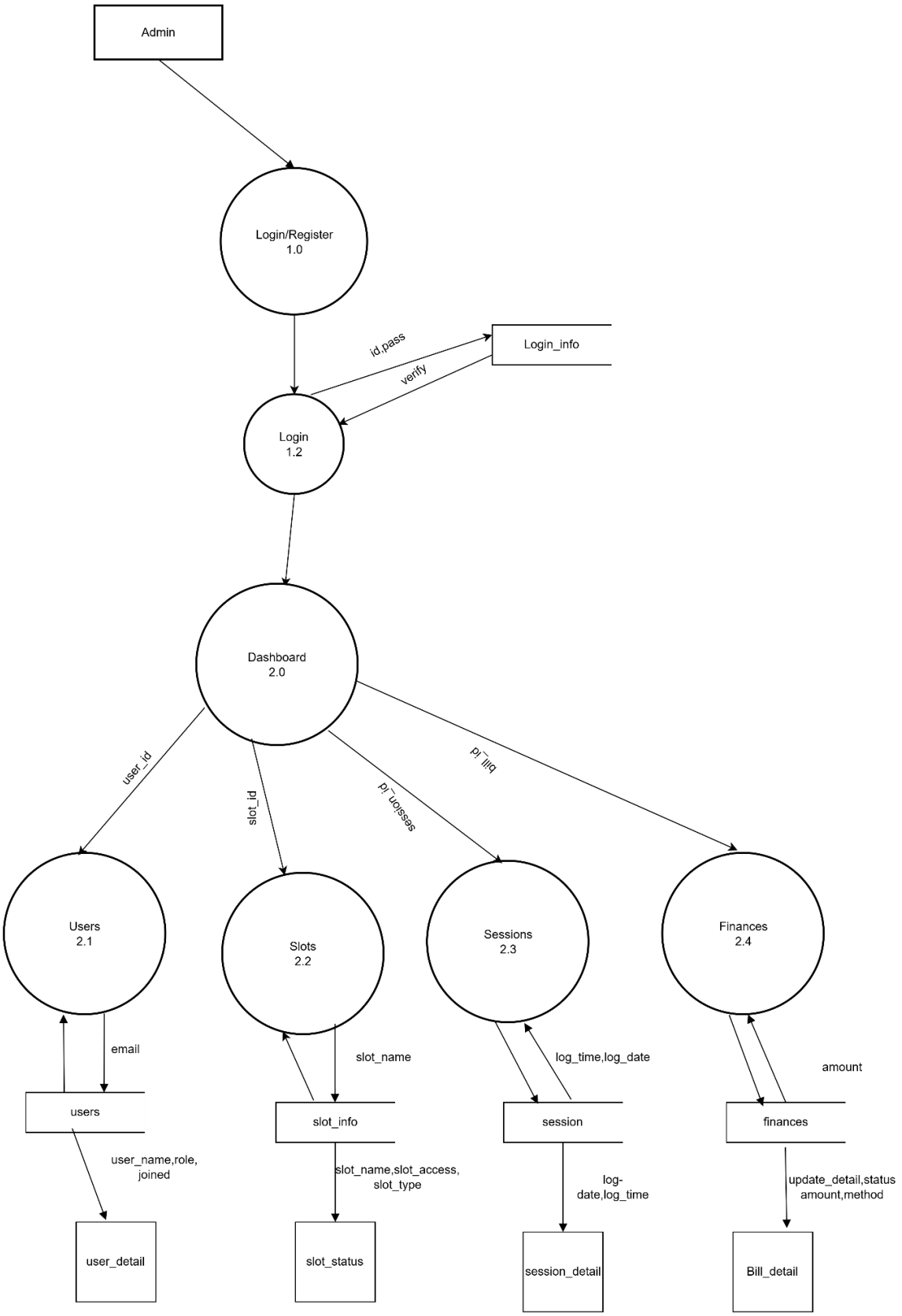
S. No	Symbol	Name	Description
1		Source/Destination	Represents a person or system that gives data to the system or receives data from it.
2		Process	Represents a process or activity that takes input data, processes it, and produces meaningful output.
3		Table	Represents a place where data is saved and stored.
4		Direction/Flow	Represents the direction of data flow between different parts of the system.

2.3 Example

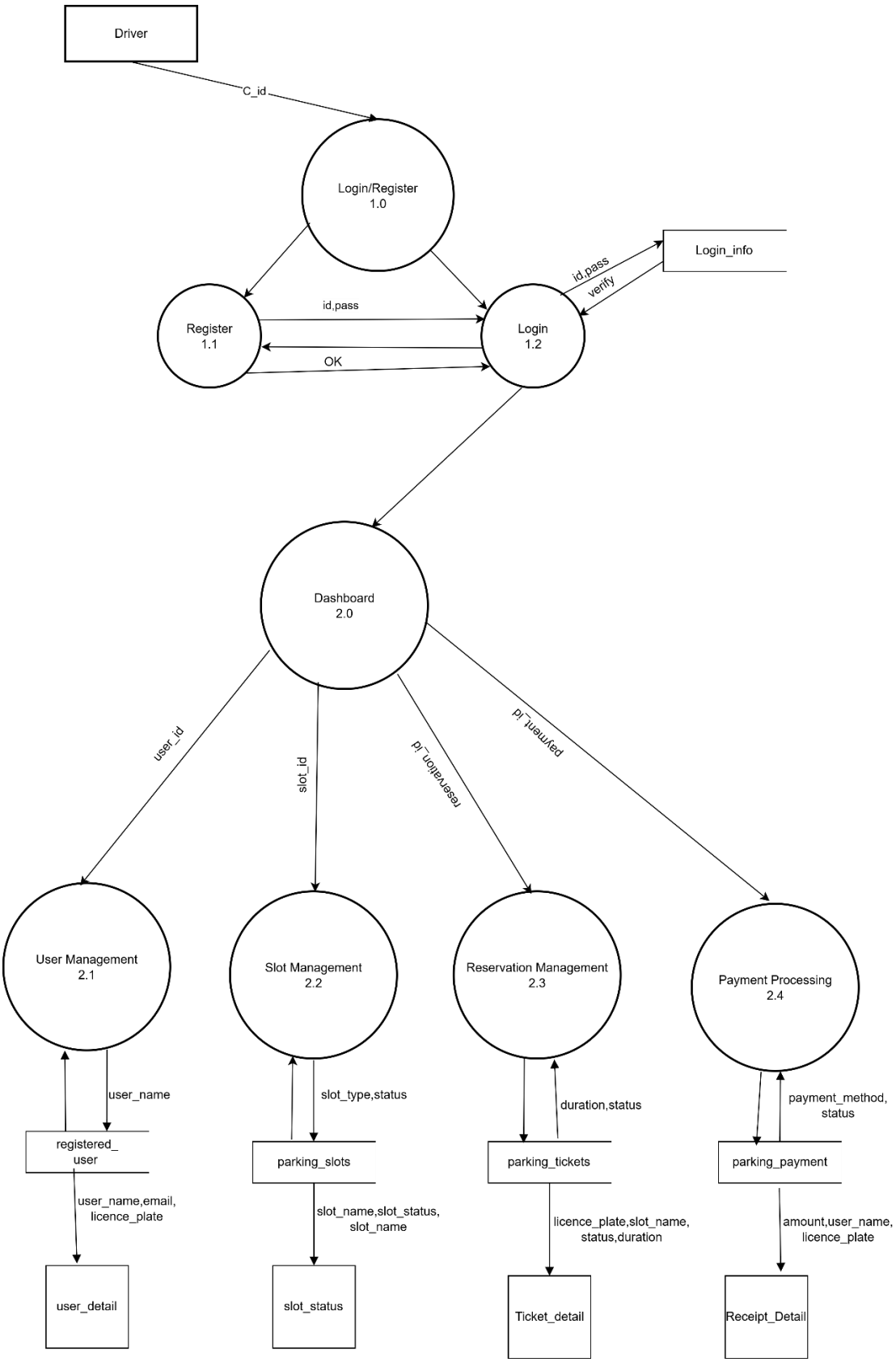
2.3.1 DFD Level 0



2.3.2 DFD Level 1 for Admin



2.3.3 DFD Level 1 For Driver

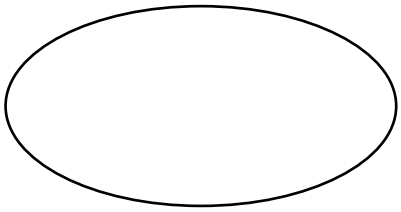
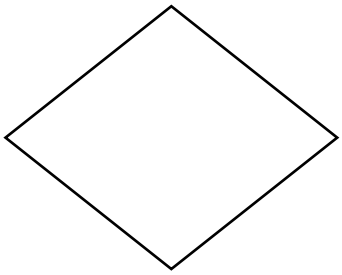
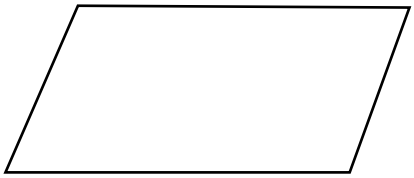


3. Flow Chart

3.1 Definition

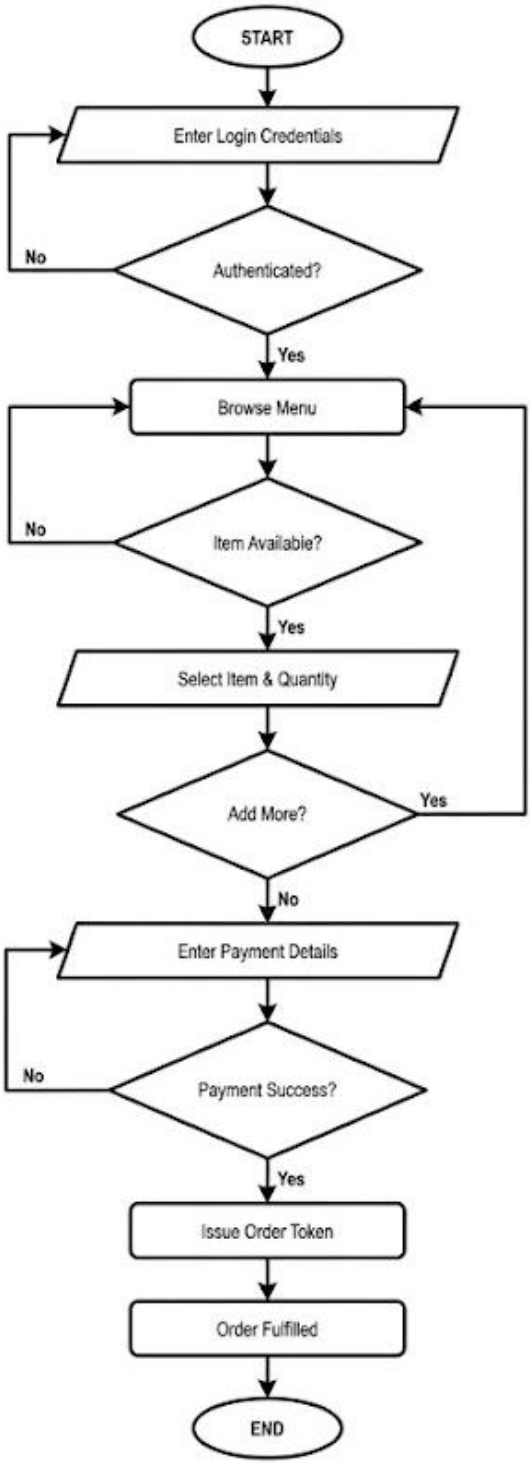
A flowchart is a graphical representation of a process or algorithm that helps in understanding the sequence of steps visually. It uses standardized symbols connected by arrows to depict the flow of control from start to end.

3.2 Symbols Used

S. No	Symbol	Name	Description
1		Terminal/Terminator	Represents the beginning or end of a process, indicating where the flow starts or stops.
2		Process	Represents a step where an action, operation, or calculation is performed.
3		Decision	Represents a point where a condition is checked, resulting in different possible paths such as Yes or No.
4		Data or Input/Output	Represents the input of data into the system or the output of information from the system.

5		Flow Arrow	Shows the direction of flow in a process, connecting different steps in the flowchart.

3.3 Example



4. Unified Modeling Language (UML)

4.1 Definition

UML (Unified Modeling Language) is a standard visual language used to design and represent software systems using diagrams. It helps in understanding both the structure and the behavior of a system by providing a set of standardized notations for various aspects of software design.

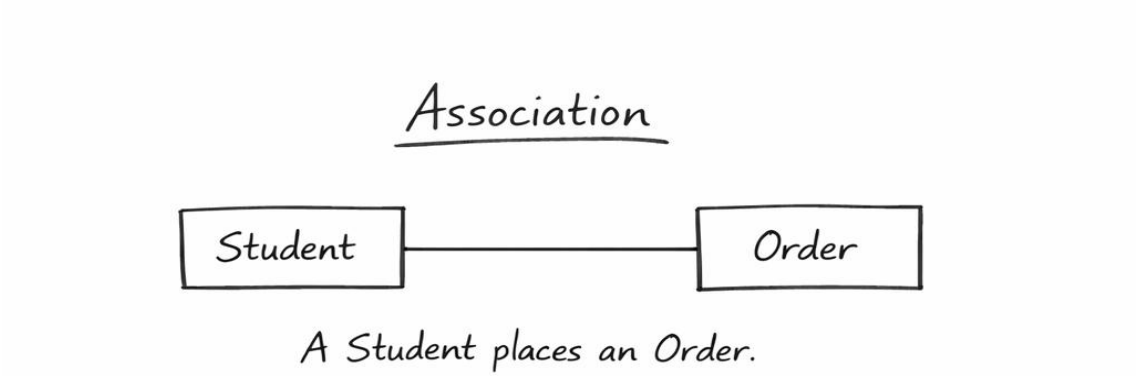
4.2 Types of UML Relationships

4.2.1 Association:

Association is a simple relationship between two classes that shows how they are connected or interact with each other. It represents a "uses" relationship and does not involve ownership.

Symbol:

Example:

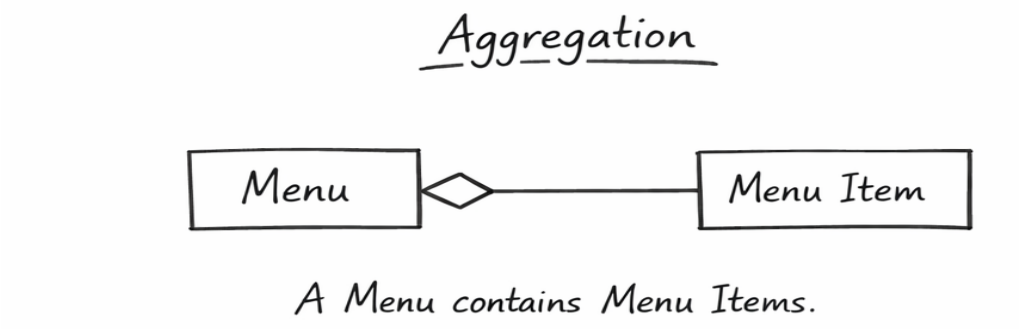


4.2.2 Aggregation:

Aggregation is a weak "has-a" relationship where one class contains another, but both can exist independently.

Symbol:

Example:

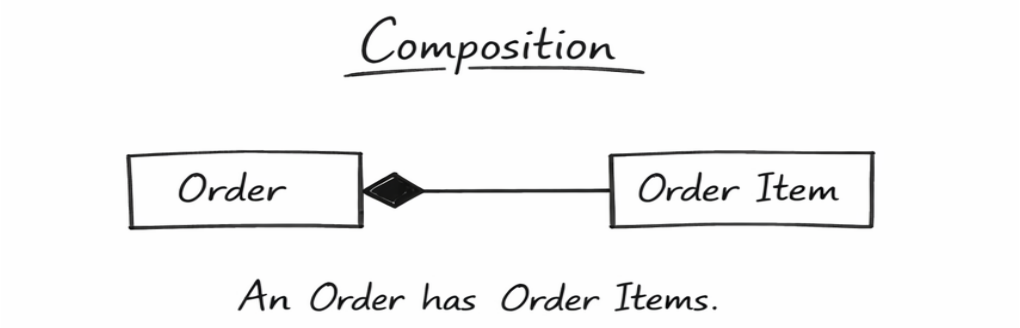


4.2.3 Composition

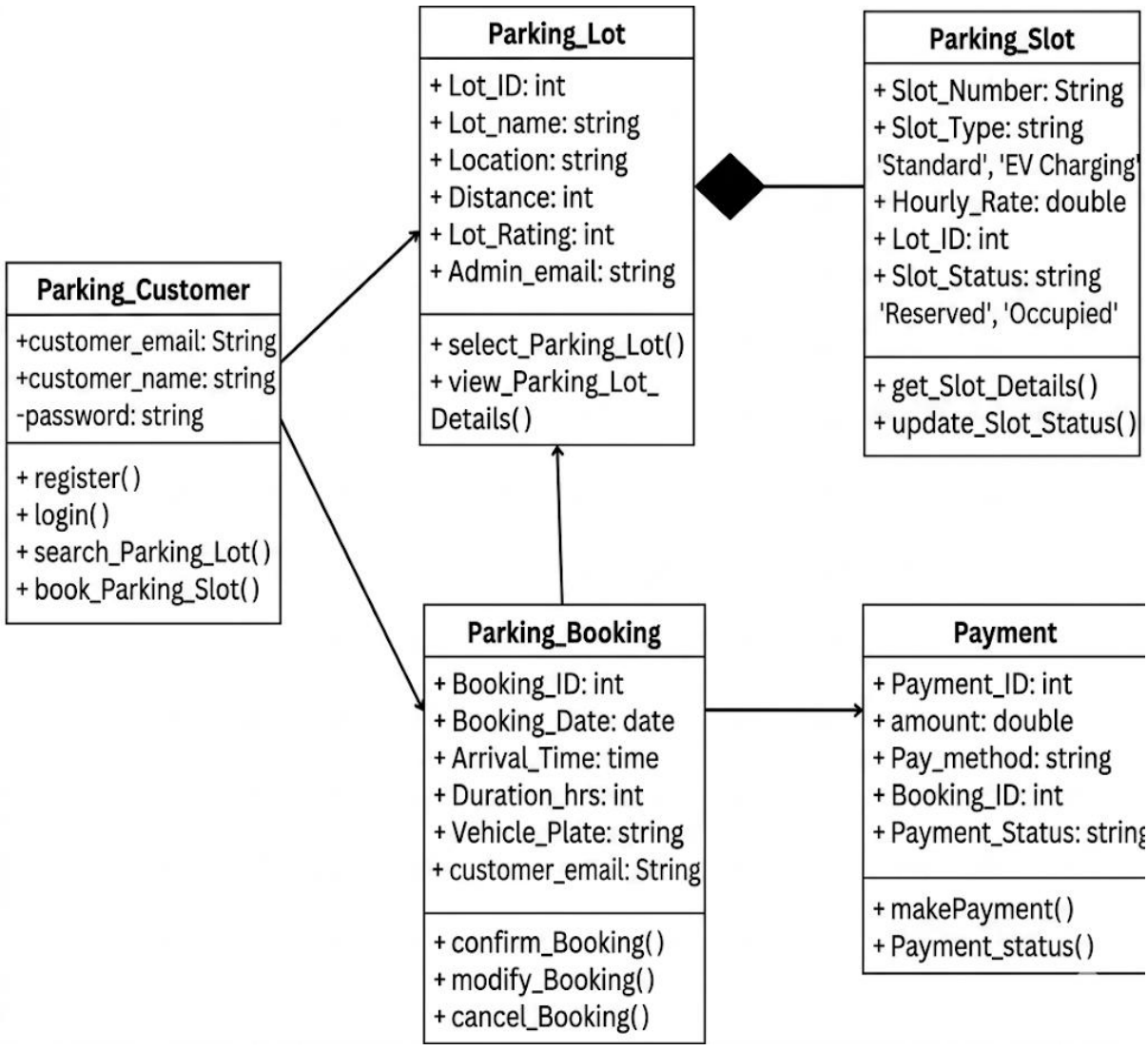
Composition is a strong “has-a” relationship where one class fully owns another, and the part cannot exist without the whole.

Symbol:

Example:



4.3 Class Diagram



5. ENTITY - RELATIONSHIP (ER) DIAGRAM\

5.1 Definition

An Entity Relationship Diagram (ERD) is a graphical tool used in database design to represent the structure of data and the relationships between different entities. It helps in visually understanding how data is organized, stored, and connected within a system.

5.2 Components of ER Diagram

5.2.1 Entity

An entity represents any real-world object about which data is stored, such as a Student or Order.

Strong Entity: Has its own unique identity (e.g., Student)

Weak Entity: Depends on another entity for identification (e.g., OrderItem)

5.2.2 Attributes

Attributes describe the details or characteristics of an entity.

Simple: Cannot be further divided (Age)

Composite: Can be split into parts (Name → First Name, Last Name)

Multivalued: Can have more than one value (Phone Numbers)

Derived: Calculated from other attributes (Total Bill)

Key Attribute: Uniquely identifies each record (ID)

5.2.3 Relationship

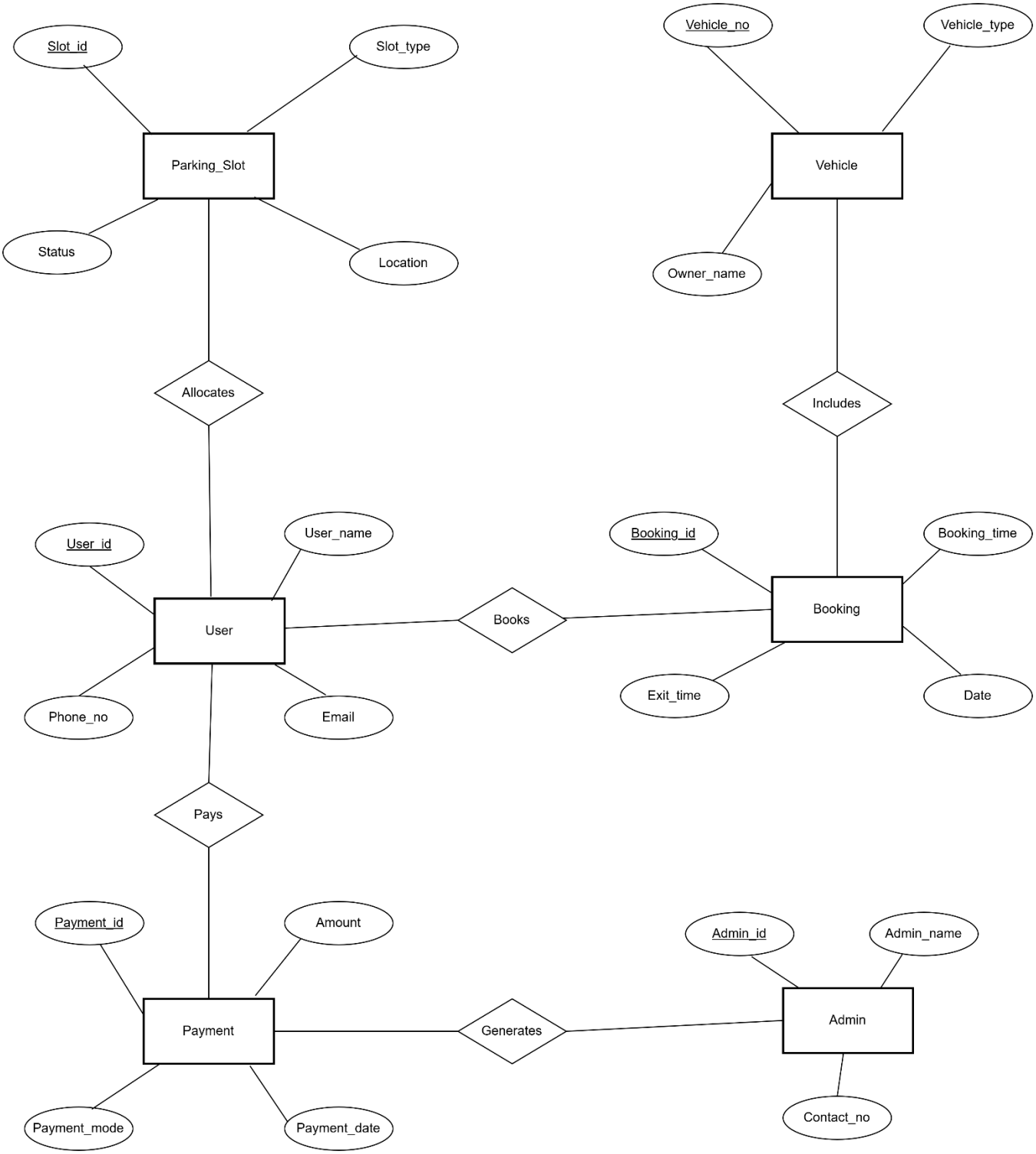
A relationship shows how different entities are connected within the system.

One-to-One (1:1): One entity is linked to one entity

One-to-Many (1:M): One entity is linked to multiple entities

Many-to-Many (M:N): Multiple entities are linked to multiple entities

5.3 Example


Slot_id
Slot_type
Parking_Slot
Status
Location
Vehicle_no
Vehicle_type
Vehicle
Owner_name
Allocates
Includes
User_id
User_name
User
Phone_no
Email
Books
Booking_id
Booking_time
Booking
Exit_time
Date
Pays
Payment_id
Amount
Payment
Payment_mode
Payment_date
Generates
Admin_id
Admin_name
Admin
Contact_no

6. TEST CASE

6.1 Definition

A test case is a defined set of conditions, inputs, and steps used to check whether a software system is working correctly. It includes input data, expected results, and actual results, which help in verifying if the system meets the required specifications. Test cases are useful for identifying errors and ensuring proper system behavior.

6.2 Components

a. Test ID

It is a unique identification number or code given to each test case. It helps in easily tracking and managing test cases. Example: TC01, TC02.

b. **Input Data**

These are the values or conditions given to the software during testing. Input data is used to perform the test. Example: Username = “Admin”, Password = “1234”.

c. **Expected Output**

This is the result that should be produced if the system is working correctly. It is defined before testing starts.

d. Actual Output

This is the result that is actually produced when the test case is executed in the system.

6.3 Example

Test Case ID	Section	Element	Test Data	Expected Result	Actual Result
1	Admin Login	Username, Password	No Data	Show Fill all Field	Passed

			admin@123 / wrong	Invalid Credentials	Passed
			admin@parking / 1234	Login Successful	Passed
2	User Login	Email, Password	Empty Feilds	Show Error	Passed
			surya@gmail / 1234	Invalid Email	Passed
			surya@gmail.com / 1234	Login Successful	Passed
3	Registration	Name, Email, Phone	Empty data	Error Message	Passed
			Invalid Email Format	Invalid Detail	Passed
			View Details	Successfully Registered	Passed
4	Slot Booking	Slot Type, Date/Time	No Slot Selected	Show Warning	Passed
			Past Date Entered	Invalid Input	Passed
			Valid Slot + DateTime	Booking Confirmed	Passed
5	Payment	Payment Method	No Selection	Show Error	

			Invalid Payment	Failed Transaction	Passed
			Valid Payment	Payment Successfully	Passed

7. CONCLUSION

The development of the Smart Parking Management System has successfully addressed the widespread challenges associated with traditional manual parking operations. By leveraging a modern technology stack such as the MERN stack (MongoDB, Express, React, and Node.js), the system provides a smooth, responsive, and real-time experience accessible on both desktop and mobile platforms.

With features like automated slot allocation, QR-based vehicle entry and exit, real-time occupancy tracking, and digital billing, the system eliminates common pain points including traffic congestion at entry gates, manual billing errors, and lack of coordination between parking staff and drivers. The integration of a feedback mechanism also enables continuous service improvement based on user experience.

The system's admin module ensures complete visibility over daily operations, providing the management with accurate reports for better decision-making. The multi-role access control guarantees that Drivers, Operators, and Administrators each interact with only the functionalities relevant to their responsibilities.

In conclusion, this project significantly contributes to a more organized, efficient, and user-friendly parking ecosystem. Future enhancements may include integration with city-wide parking networks, AI-based demand forecasting, license plate recognition for automatic vehicle identification, and IoT-based sensor integration for real-time slot status detection.